

Aula 06

Arquitetura em Camadas

Objetivos

Organizar o projeto em camadas, separando responsabilidades e facilitando a manutenção e evolução do sistema.

Refatorar a tela inicial do sistema para preparar para a implementação de relacionamentos entre entidades, garantindo que a arquitetura esteja adequada para suportar futuras funcionalidades.

Preparar para o Menu de Navegação, que será implementado na próxima aula, garantindo que a estrutura do projeto esteja pronta para receber essa funcionalidade.

Preparação para o React Router, que será utilizado para gerenciar a navegação entre diferentes páginas do sistema, garantindo uma experiência de usuário fluida e consistente.

Revisão da Aula Anterior

Completamos o CRUD de Usuario : POST , GET , PUT e DELETE na classe Api

Criamos formulários controlados em React (UsuarioForm) com useState

UsuarioList consome a API via Axios e recarrega a lista após cada operação

App.tsx renderiza a tela de listagem diretamente

Problema: está tudo **concentrado** — o Api (controller) acessa o banco direto, e o React não tem uma organização que suporte o crescimento do sistema. Hoje vamos organizar em **camadas**.

Conceitos Fundamentais

O que é arquitetura em camadas?

Divisão do sistema em camadas com **responsabilidades bem definidas**

Cada camada conversa apenas com a camada vizinha (fluxo de cima para baixo)

Facilita **manutenção, testes, reuso e evolução**

Camadas no Spring Boot

Camada	Responsabilidade	Anotação
Controller	Receber requisições HTTP e devolver respostas	@RestController
Service	Regras de negócio e validações	@Service
Repository	Acesso e persistência no banco	interface JpaRepository

Controller não fala com Repository direto — quem coordena é o Service.

Por que separar Controller e Service?

Antes (Aula 05): o Api fazia **tudo** — recebia a requisição e acessava o banco.

```
@GetMapping("/usuarios")  
public List<Usuario> getAllUsuarios() {  
    return usuarioRepository.findAll(); // controller acessa o banco direto }
```

Problemas:

- Regras de negócio misturadas com detalhes de HTTP

- Difícil **testar** a lógica sem subir o servidor

- Ao adicionar validações/permissions, o controller vira um "canivete suíço" Com o

- Service**, o controller apenas traduz HTTP e o Service concentra as regras. 5

Frontend
(Cliente)



React

(frontend)

http://localhost:5173

ID	Nome	Email	Ações
1	Ana Souza	ana@email.com	✎ ✖
2	João Silva	joao@email.com	✎ ✖

- Interface do usuário
- Envia requisições HTTP usando Axios
- Consome a API (JSON)
- Exibe e manipula os dados



Requisição HTTP (JSON)

GET /api/usuarios
POST /api/usuarios
PUT /api/usuarios/{id}
DELETE /api/usuarios/{id}



Resposta (JSON)

```
[{
  "id": 1,
  "nome": "Ana Souza",
  "email": "ana@email.com"
}]
```

- Axios faz requisições para a API
- Envia e recebe dados em formato JSON
- Métodos HTTP: GET, POST, PUT, DELETE

Camada de Controle
(Controller)



UsuarioController

(@RestController)

@RequestMapping("/api/usuarios")

```
GET    /api/usuarios
POST   /api/usuarios
PUT    /api/usuarios/{id}
DELETE /api/usuarios/{id}
```

- Recebe as requisições HTTP
- Traduz HTTP para chamadas de método
- Retorna respostas em JSON
- Atua como ponto de entrada da API

Camada de Serviço
(Service)



UsuarioService

(@Service)

```
listarUsuarios()
buscarPorId(id)
criarUsuario(usuario)
atualizarUsuario(id, usuario)
deletarUsuario(id)
```

- Contém as regras de negócio
- Validz dados
- Orquestra operações
- Chama o repositório para acessar os dados

Camada de Persistência
(Repository)



UsuarioRepository

(JpaRepository)

```
interface UsuarioRepository
extends JpaRepository<Usuario, Long>
{
  • findAll()
  • findById()
  • save()
  • deleteById()
}
```

- Interface que estende JpaRepository
- Fornece métodos prontos de acesso a dados
- Conecta com o banco via Spring Data JPA
- Não contém regras de negócio

Camada de Dados
(Banco de Dados)



Banco H2

(em memória)

Tabela: usuarios

- id (PK)
- nome
- email

- Banco de dados H2 (em memória)
- Armazena os dados da aplicação
- Utilizado para desenvolvimento e testes
- Pode ser acessado via console do H2

Organização de pacotes no Spring Boot

Antes — tudo concentrado no pacote raiz:

```
br.ueg.trindade.braully_web2_fullstack/  
├── Api.java  
├── Usuario.java  
├── UsuarioRepository.java  
└── BraullyWeb2FullstackApplication.java
```

Reorganizar os pacotes para uma arquitetura em camadas

Depois — cada classe no pacote da sua camada:

```
br.ueg.trindade.braully_web2_fullstack/  
├── BraullyWeb2FullstackApplication.java  
├── controller/  
│   ├── UsuarioController.java (@RestController)  
├── service/  
│   ├── UsuarioService.java (@Service)  
├── repository/  
│   ├── UsuarioRepository.java (JpaRepository)  
└── model/  
    ├── Usuario.java (@Entity)
```

IMPORTANTE: o Spring Boot **não exige** essa organização — é apenas uma **boa prática** para projetos maiores.

Organização de pastas no React

Antes — componentes soltos em `components/` :

```
frontend/src/  
├── components/  
│   ├── UsuarioForm.tsx  
│   ├── UsuarioItem.tsx  
│   └── UsuarioList.tsx  
├── services/  
│   └── api.ts  
├── types/  
│   └── Usuario.ts  
├── App.tsx  
└── main.tsx
```

Vamos separar **telas** (pages/) de **componentes reutilizáveis** (components/), preparando o terreno para o **React Router** (Aula 08).

Programação Web II

Organização de pastas no React (depois)

```
frontend/src/
├── components/ # peças reutilizáveis
│   ├── UsuarioForm.tsx
│   └── UsuarioItem.tsx
├── pages/ # telas do sistema (rotas)
│   └── UsuariosPage.tsx
├── services/ # chamadas HTTP
│   └── api.ts
├── types/ # tipos TypeScript
│   └── Usuario.ts
├── App.tsx # em breve: define as rotas
└── main.tsx
```

Regra prática: se é uma *tela* inteira, vai em `pages/` ; se é um *pedaço reutilizável* (botão, formulário, card), vai em `components/` .

Demonstração — Refatorando o back-end

Passo 1: criar o `UsuarioService`

```
@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    public List<Usuario> listarTodos() {
        return usuarioRepository.findAll();
    }

    public Usuario buscarPorId(Long id) {
        return usuarioRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));
    }
}
```

O @Service indica ao Srin ue essa classe é um comonente da camada
Programação Web II

Demonstração — UsuarioService (completo)

```
@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    public List<Usuario> listarTodos() {
        return usuarioRepository.findAll();
    }

    public Usuario buscarPorId(Long id) {
        return usuarioRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Usuário não encontrado"));
    }

    public Usuario criar(Usuario usuario) {
        return usuarioRepository.save(usuario);
    }

    public Usuario atualizar(Long id, Usuario usuarioAtualizado) {
        Usuario usuario = buscarPorId(id);
        usuario.setNome(usuarioAtualizado.getNome());
        usuario.setUsername(usuarioAtualizado.getUsername());
        usuario.setEmail(usuarioAtualizado.getEmail());
        return usuarioRepository.save(usuario);
    }
}
```

```
public void excluir(Long id) {  
    usuarioRepository.deleteById(id);  
}
```

Programação Web II

Demonstração — Passo 2: criar o UserController

```
@RestController  
@RequestMapping("/api/usuarios")  
@CrossOrigin(origins = "http://localhost:5173")  
public class UserController {
```

```
    @Autowired  
    private UsuarioService usuarioService;
```

```
    @GetMapping  
    public List<Usuario> getAllUsuarios() {  
        return usuarioService.listarTodos();  
    }
```

```
    @GetMapping("/{id}")
```

```
public Usuario getUsuarioById(@PathVariable Long id) {  
    return usuarioService.buscarPorId(id);  
}
```

@PostMapping

```
public Usuario createUsuario(@RequestBody Usuario usuario) {
```

Programação Web II

14

Demonstração — UsuarioController

```
@RestController  
@RequestMapping("/api/usuarios")  
@CrossOrigin(origins = "http://localhost:5173")  
public class UsuarioController {  
  
    @Autowired  
    private UsuarioService usuarioService;  
  
    @GetMapping  
    public List<Usuario> getAllUsuarios() {  
        return usuarioService.listarTodos();  
    }  
  
    @GetMapping("/{id}")  
    public Usuario getUsuarioById(@PathVariable Long id) {  
        return usuarioService.buscarPorId(id);  
    }  
  
    @PostMapping  
    public Usuario createUsuario(@RequestBody Usuario usuario) {  
        return usuarioService.criar(usuario);  
    }  
}
```

```

@PutMapping("/{id}")
public Usuario updateUsuario(@PathVariable Long id, @RequestBody Usuario usuarioAtualizado) {
    return usuarioService.atualizar(id, usuarioAtualizado);
}

```

```

@DeleteMapping("/{id}")
public void deleteUsuario(@PathVariable Long id) {
    usuarioService.excluir(id);
}

```

```

}

```

ProgramaçãoWebII

15

Demonstração—Refatorandoofront-end

Passo1:criarpages/UsuariosPage.tsx

A lógicadeUsuarioListviraumapágina(ateladeusuários):

```

// src/pages/UsuariosPage.tsx
import { useEffect, useState } from "react";
import api from "../services/api";
import type { Usuario } from "../types/Usuario";
import UsuarioItem from "../components/UsuarioItem";
import UsuarioForm from "../components/UsuarioForm";

function UsuariosPage() {
    const [usuarios, setUsuarios] = useState<Usuario[]>([]);
    const[editando,setEditando]=useState<Usuario | null>(null);

    functioncarregarUsuarios(){

```

```

    api.get<Usuario[]>("/usuarios").then((resposta)=>{
      setUsuarios(resposta.data);
    });
  }

  useEffect(()=>{
    carregarUsuarios();
  }, []);

  async function excluir(id: number){
    await api.delete(`/usuarios/${id}`);
    carregarUsuarios();
  }
}

```

Programação Web II

16

Demonstração — UsuariosPage.tsx (render)

```

return (
  <div>
    <h1>Usuários</h1>

    <UsuarioForm
      key={editando?.id ?? "novo"}
      usuarioEditando={editando}
      onUsuarioSalvo={() => {
        carregarUsuarios();
        setEditando(null);
      }}
    />
  </div>
)

```

```

    />

    <ul>
      {usuarios.map((usuario) => (
        <li key={usuario.id}>
          <UsuarioItem usuario={usuario} />
          <button onClick={() => setEditando(usuario)}>Editar</button>
          <button onClick={() => excluir(usuario.id)}>Excluir</button>
        </li>
      ))}
    </ul>
  </div>
);
}

```

```

export default UsuariosPage;

```

Programação Web II

17

Demonstração — Passo 2: atualizar o App.tsx

```

import UsuariosPage from "../pages/UsuariosPage";

```

```

function App() {
  return <UsuariosPage />;
}

```

```
export default App;
```

App.tsx deixa de conter a lógica da tela — apenas **monta a página** Na Aula 08, ele passará a definir as **rotas** com React Router

A estrutura em `pages/` já deixa tudo pronto para o **Menu de Navegação** 18

Programação Web II

Atividade Prática

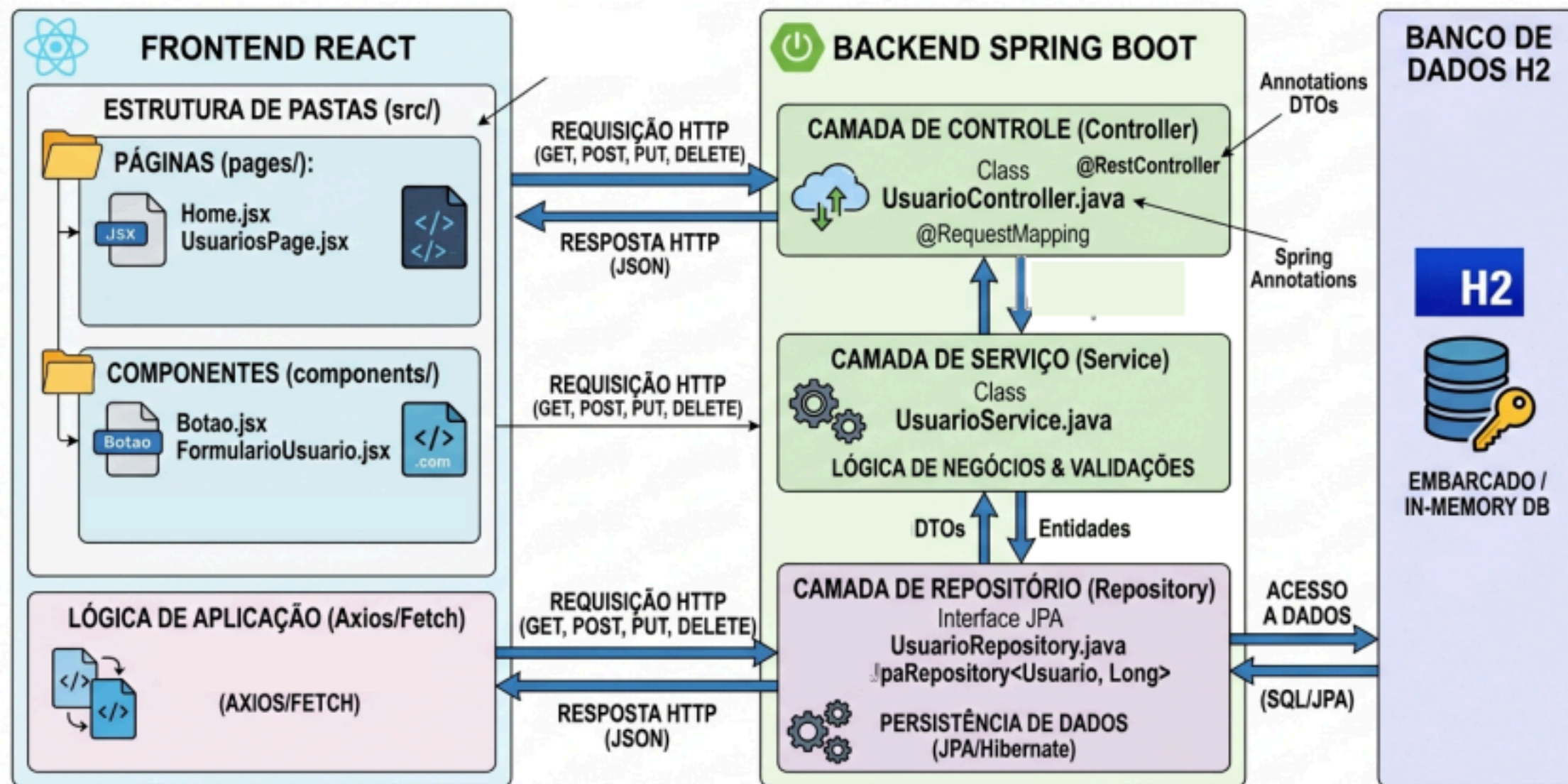
Objetivo

Refatorar o projeto para arquitetura em camadas: criar `UsuarioService` no back end e organizar as pastas do React (`pages/` × `components/`).

Passos

1. Back-end: criar UsuarioService com listarTodos , buscarPorId , criar , atualizar e excluir
2. Back-end: criar UsuarioController (novo) que injeta o UsuarioService e **não** acessa mais o Repository
3. Remover/renomear a antiga classe Api — o CRUD agora vive no controller + service

DIAGRAMA DA ARQUITETURA COMPLETA FULLSTACK (React + Spring Boot + H2)



Exercícios

1. Repetir a refatoração para Permissao : criar PermissaoService + PermissaoController e uma PermissoesPage no React
2. Refatorar a entidade do seu interesse (Produto , Pedido , Cliente , etc.) para o padrão Controller → Service → Repository
3. Adicionar uma **regra de negócio** no Service da sua entidade (ex.: impedir cadastro com e-mail duplicado) e validar no navegador
4. Ao final, comitar e subir as alterações no repositório do GitHub

Referências

Spring Boot Documentation

Spring — Building a RESTful Web Service: <https://spring.io/guides/gs/rest-service/>

Baeldung — Controller, Service, Repository:

<https://www.baeldung.com/spring-component-repository-service>

React Documentation — <https://react.dev/learn>

Material complementar da disciplina

Próxima Aula

Conteúdo: Relacionamentos com JPA — @OneToMany , @ManyToOne , @ManyToMany e DTOs

Preparação: deixar o projeto refatorado em camadas (Controller → Service → Repository) com o CRUD de Usuario funcionando